

Personal Expense Ledger & Social Debt Roadmap

A practical architecture for tracking a strict \$800 personal spending limit when credit cards, shared groceries, cash reimbursements, and friend IOUs are all mixed together.

Budget-first **CSV MVP** **Splitwise later** **PWA-ready** **AI voice later**

1. Executive Summary

This project should not be treated as a normal budget tracker. Your credit card statement shows **cash flow**, while your monthly \$800 limit should track **personal consumption**. These are different things.

Core thesis: Transactions are payment events. Allocations are ownership of consumption. Settlements are debt-clearing events. Your budget is calculated only from your personal allocations.

The app should let you import card transactions quickly, leave them in an inbox, and split them later when you have time. It must also handle the reverse case where someone else paid for something that belongs to you.

2. The Real-World Problem Model

What appears on your card

- Personal spending
- Groceries bought for the house
- Items bought for someone else
- Refunds and adjustments
- Full shared bills that are not fully yours

What your budget should count

- Your share of groceries
- Your meals, coffee, transit, subscriptions
- Your share of expenses paid by someone else
- Manual cash expenses
- Not the full shared bill

The app must preserve the difference between **paid by me**, **consumed by me**, **owed to me**, and **owed by me**.

3. Non-Negotiable Product Principles

Inbox first

You should be able to import transactions without splitting immediately. Busy life is a product requirement.

Budget from allocations

The \$800 tracker must sum only your personal allocation lines, not raw credit card totals.

Manual entries matter

If someone else paid for you, it must still count against your budget even though it never touched your card.

Settlements are first-class

Cash, e-transfer, and offset repayments must be tracked explicitly rather than hidden as notes.

Splitwise is not the ledger

Your local ledger should be the source of truth. Splitwise is a communication/sync layer.

AI is just an input method

Voice/AI should call the same backend services as the UI.

4. Recommended Architecture

For a solo project, use a **Next.js PWA + PostgreSQL + Prisma/Drizzle** architecture. It keeps the web UI, backend routes, mobile-friendly app, and database access in one maintainable codebase.

Next.js PWA
Dashboard | Inbox | Split Modal | Balances | Mobile Quick Add
|
v
Application Services
Import | Allocation | Budget | Settlement | Balance | Splitwise Sync | AI Parse
|
v
PostgreSQL
transactions | allocations | settlements | people | groups | categories | impor
|
v
External Integrations
Splitwise API | Plaid/Open Banking later | Speech-to-text/LLM later

5. Recommended Tech Stack

LAYER	CHOICE	REASON
Frontend + backend	Next.js	One codebase for UI, API routes/server actions, PWA, and deployment simplicity.
Database	PostgreSQL	Reliable relational model for ledger-style data, splits, settlements, and reporting.
ORM	Prisma or Drizzle	Prisma is productive; Drizzle is lighter and more SQL-like.
Deployment	Vercel + Neon/Supabase Postgres	Simple hosted setup for a personal app.
Mobile	PWA first	Phone-friendly without native app overhead.

AI voice

Speech-to-text + LLM
JSON parser later

Voice becomes a thin input layer over
stable APIs.

6. Core Data Model

The database should support three separate concepts: payment events, consumption ownership, and debt settlement.

transactions

```
id
transaction_date
description
merchant
amount
currency
source_type
payment_account_id
import_id
external_id
status
notes
created_at
```

allocations

```
id
transaction_id
person_id
amount
category_id
allocation_type
budget_impact
splitwise_sync_status
```

```
splitwise_expense_id  
created_at
```

settlements

```
id  
settlement_date  
from_person_id  
to_person_id  
amount  
currency  
method  
notes  
created_at
```

people

```
id  
display_name  
is_me  
splitwise_user_id  
email  
created_at
```

Other important tables

- **accounts:** credit cards, cash wallet, bank accounts, manual accounts.
- **categories:** groceries, eating out, transit, household, entertainment.
- **budget_periods:** start date, end date, limit amount, currency.
- **imports:** CSV metadata, source, timestamp, row counts.
- **sync_events:** Splitwise sync attempts, statuses, errors, retries.
- **groups/group_members:** useful for housemates and recurring splits.

7. Budget and Balance Rules

Budget calculation: monthly personal spend = sum of allocations where person is you and budget_impact is true within the budget period.

```
monthly_personal_spend = SUM(allocations.amount)
WHERE allocations.person_id = Darshan
AND allocations.budget_impact = true
AND allocation_date BETWEEN budget_period.start_date AND budget_period.end_date
```

METRIC	MEANING
Personal spending	What you consumed and should count toward \$800.
Card cash flow	What hit your credit card or cash account.
People owe me	Receivables created when you paid for others.
I owe others	Payables created when someone else paid for you.
Unreviewed exposure	Imported transactions not allocated yet.

8. Key User Flows

You paid for shared groceries

1. CSV import creates a \$120 Walmart transaction.
2. Transaction appears in inbox.
3. Later, allocate \$45 to yourself, \$40 to Logan, \$35 to Rahul.

4. Your budget increases by \$45 only.
5. Others' balances show amounts owed to you.
6. Optional: push to Splitwise.

Someone else paid for you

1. Manual entry: Rahul paid \$32 dinner, your share \$16.
2. App creates a payable to Rahul.
3. Your budget increases by \$16.
4. No credit card transaction is required.

Cash reimbursement

1. Logan owes you \$40.
2. He gives you \$25 cash.
3. Record settlement: Logan to Darshan, \$25, cash.
4. Remaining balance: Logan owes \$15.
5. Your \$800 budget does not change.

Offset settlement

1. You owe Ankit \$20.
2. Ankit owes you \$30.
3. App shows net: Ankit owes you \$10.
4. Record offset or show net balance.

9. API Surface for MVP

```
POST    /api/imports/csv
GET      /api/transactions?status=needs_review
POST    /api/transactions/manual
GET      /api/transactions/:id
POST    /api/transactions/:id/allocations
PATCH  /api/allocations/:id
DELETE  /api/allocations/:id
GET      /api/budget/current
GET      /api/balances/people
POST    /api/settlements
GET      /api/settlements
POST    /api/splitwise/sync/:allocationId    # later
POST    /api/ai/parse-command                # later
```

10. Phased Roadmap

Phase 0 - Ledger validation

Goal Prove the model handles messy life before building too much UI.

- Write sample scenarios: shared groceries, cash repayment, someone paid for you, refund, partial settlement, offset, month boundary.
- Confirm each scenario can be represented by transactions, allocations, and settlements.
- Decide default categories and budget period.

Exit: You can manually calculate correct personal spend and balances from the model.

Phase 1 - Local MVP

- Create Next.js project with Postgres and ORM.
- Create tables: people, accounts, transactions, allocations, categories, budget_periods, imports.
- Build CSV import with duplicate detection.
- Build transaction inbox and allocation modal.
- Build current month budget dashboard.
- Add manual transaction entry.

Exit: Upload a statement, allocate later, and see true monthly spend.

Phase 2 - Settlements and balances

- Add settlements table and UI.
- Support cash, e-transfer, card, and offset settlements.
- Build people balance page.
- Support partial settlements and corrections.

Exit: You can answer who owes whom at any time.

Phase 3 - Splitwise sync

- Add Splitwise identity mapping.
- Add sync_events table.
- Implement create expense for selected allocations.
- Store external IDs, statuses, errors, retry data.

Exit: Local splits can be pushed safely and retried.

Phase 4 - Mobile-first PWA

- Responsive dashboard and inbox.
- Quick add manual expense flow.
- Add to Home Screen support.

- Optional offline draft entry.

Exit: Most quick events can be recorded from your phone.

Phase 5 - AI voice input

- Use phone speech-to-text or transcription.
- Parse natural language into structured JSON.
- Show confirmation before saving.
- Support commands like "Logan paid me \$40 cash".

Exit: Common expense and settlement commands can be captured quickly.

Phase 6 - Bank sync only if still needed

- Evaluate Plaid/open banking options carefully.
- Implement account linking only after security tradeoffs are acceptable.
- Use sync only to populate the same inbox.
- Keep CSV as fallback.

11. What Not to Build Too Early

Do not start with Plaid

Bank sync adds auth, security, consent, webhook, and cost complexity. CSV is enough to validate.

Do not start with native mobile

A PWA likely solves the phone need first.

Do not let AI mutate data blindly

AI should draft structured entries and ask for confirmation.

Do not make Splitwise the ledger

Your budget logic should live locally.

12. Success Metrics

- Upload a credit card CSV and deduplicate repeat imports.
- Leave transactions unreviewed without breaking the budget.
- Allocate one transaction across yourself and multiple people.
- Record expenses someone else paid for you.
- Record cash/e-transfer settlements.
- See per-person net balances.
- Use the app weekly without creating more work than it saves.

13. Final Recommendation

Build this, but build the ledger foundation first. The project becomes valuable as soon as it can answer: **How much have I personally spent this month, and who**

owes whom?

Everything else - Splitwise, AI, mobile, and automatic bank import - should be layered on top of that stable foundation.

Generated roadmap for Darshan Jain - Personal Expense Ledger Project